

Gateway E2E Automation Framework — KT Guide

Product: Services Gateway (GlobalPayments Gateway API)
Framework: `GlobalPayments.Gateway.Client.Tests.End2End`
Author: Brock Pivrotto
Last Updated: August 2026

Overview

The Gateway E2E framework is a C# MSTest REST API automation suite that tests the **Services Gateway** — Pavilion's internal gateway layer that translates operator-facing calls into downstream banking network requests. It is structurally similar to the VIP Connect API framework but differs in how authentication works and how test data is managed.

Attribute	Gateway E2E	VIP Connect API
Language / Runtime	C# / .NET 10	C# / .NET 8
Test Runner	MSTest	MSTest
HTTP Client	RestSharp	RestSharp
Auth Method	Azure AD S2S (client credentials OAuth2)	Operator Bearer Token
Config File	test-keys.json + test-keys.Development.json	test-keys.json + test-keys.Development.json
Token Lifecycle	Acquired once per assembly run	One static token per operator
Test Data	TestDataGenerator builder methods	generatePatronSessionJson + SessionType enum
Client Lifecycle	New RestClient per test class	Static RestClient

Part 1: Project Structure

```

GlobalPayments.Gateway.Client.Tests.End2End/
├── GlobalPayments.Gateway.Client.Tests.Automation.E2E.csproj
├── MSTestSettings.cs
├── NLog.config
├── Configuration/
│   ├── TestApiKeys.cs           - Loads test-keys.json + override file
│   ├── TestEnvironmentConfig.cs - Strongly-typed config model
│   ├── test-keys.json          - Committed template (no secrets)
│   └── test-keys.Development.json - Local override with real secrets (git-
ignored)
├── Infrastructure/
│   └── BaseTest.cs              - Assembly init, token acquisition, RestClient
factory
├── TestDataGenerator.cs        - Random data generators and request builders
├── Functions/
│   ├── GatewayApiFunctions.cs  - All API calls (1:1 to Gateway endpoints)
│   └── JsonClasses/
│       ├── Requests/           - Request model classes
│       └── Responses/          - Response model classes
├── Tests/
│   ├── AccountInquiryTests.cs
│   └── API/
│       ├── EchoTests.cs / HealthTests.cs
│       ├── Accounts/          (7 test classes)
│       ├── Checks/            CheckSaleTests, CheckDisbursementTests
│       ├── Finicity/          FinicityTests
│       ├── Funding/           Bank, CreditCard, Finicity, Generic, Plaid
│       ├── Merchants/         MerchantTests, AddVipSelectTransactionTests
│       ├── Plaid/              GetPlaidTokenTests
│       └── Reports/            ActivityReportTests

```

Part 2: Authentication

2.1 How Auth Works

The Gateway uses **Azure AD Service-to-Service (S2S) OAuth2 client credentials** — not an operator bearer token. Before any test runs, `BaseTest.AssemblyInit` acquires a JWT access token from Microsoft's identity platform and stores it in the static `AccessToken` property. Every API request then includes:

```
Authorization: Bearer <access_token>
```

The token is acquired **once per full test run** at assembly initialization and reused across all test classes and methods.

2.2 Required Credentials (test-keys.Development.json)

Four fields are required for the auth flow:

Field	Description
TenantId	Azure AD tenant GUID
ClientId	Service principal / app registration client ID
ClientSecret	Client secret for the app registration
Scope	OAuth2 scope (e.g., api://xxx/.default)

If any of these are missing, `AssemblyInit` throws `InvalidOperationException` and the entire test run fails before any test executes.

2.3 IPv4 Workaround

`BaseTest` includes a `SocketsHttpHandler` with a `ConnectCallback` that forces IPv4 when resolving `login.microsoftonline.com`. This is because .NET's Happy Eyeballs algorithm prefers IPv6, which is unreachable on this network, causing token acquisition to hang. This is intentional — do not remove it.

2.4 Config Loading Order

`TestApiKeys.LoadConfig()` reads configuration in this priority order (later sources win):

1. `Configuration/test-keys.json` — committed template, placeholders only
2. `Configuration/test-keys.Development.json` — local secrets, git-ignored
3. Environment variables — used by CI/CD pipelines

Obtain `test-keys.Development.json` from the team lead or Azure Key Vault.

Part 3: Running the Tests

3.1 Prerequisites

4. Clone the repo from Azure DevOps → PAV Products → services-gateway
5. Open `services-gateway.sln` in Visual Studio 2022
6. Obtain `test-keys.Development.json` and place it at:
`GlobalPayments.Gateway.Client.Tests.End2End/Configuration/test-keys.Development.json`
7. Build the solution (Ctrl+Shift+B)

3.2 Config File Must Be in Build Output

After each build, verify the config file is present at:

```
bin\Debug\net10.0\Configuration\test-keys.Development.json
```

If missing, the test run fails with a config exception. Add it manually if the build did not copy it automatically.

3.3 Running via Visual Studio Test Explorer

8. Open **Test** → **Test Explorer** (Ctrl+E, T)
9. Build the solution to populate the test list
10. Filter by category in the search bar:
 - `Trait:"SmokeTest"` — core smoke tests only
 - `Trait:"Regression"` — full regression suite
 - `ClassName:CheckSaleTests` — all tests in one class
11. Click any result to see the NLog output

3.4 Running via Command Line

```
dotnet test GlobalPayments.Gateway.Client.Tests.End2End --logger  
"console;verbosity=normal"  
dotnet test GlobalPayments.Gateway.Client.Tests.End2End --filter  
"TestCategory=SmokeTest"  
dotnet test GlobalPayments.Gateway.Client.Tests.End2End --filter  
"ClassName~CheckSaleTests"
```

3.5 Test Categories

Category	When to Run	Applied At
SmokeTest	After every deployment	Method level
Regression	Before releases	Class level (inherited by all methods)
API	All API tests	Class level

Part 4: Framework Architecture

4.1 BaseTest

All test classes inherit from `BaseTest`. It provides:

- **AssemblyInit** — runs once before all tests; loads config and acquires Azure AD token
- **AssemblyCleanup** — runs once after all tests; shuts down NLog
- **Config** — static property holding `TestEnvironmentConfig` (all test data values)
- **AccessToken** — static string with the JWT bearer token
- **CreateGatewayClient()** — factory method returning a `RestClient` pointed at `Config.GatewayBaseAddress`

4.2 GatewayApiFunctions

`GatewayApiFunctions` is the API layer — all Gateway HTTP calls go through here. Rules:

- One static method per Gateway endpoint
- Returns `RestResponse` only — no assertions inside this class
- Every request gets a fresh `X-Activity-Id` GUID header (required by the Gateway)
- Uses `client.Execute()` (not `client.Post/Get`) so non-2xx responses return with their body instead of throwing

Usage pattern:

```
var response = GatewayApiFunctions.CheckSale(_client, AccessToken, body);
Assert.AreEqual(HttpStatusCode.OK, response.StatusCode, response.Content);
```

4.3 TestDataGenerator

`TestDataGenerator` provides:

- **Random generators** — `RandomSsn()`, `RandomEmail()`, `RandomPhoneNumber()`, `RandomDOB()`, `RandomTransactionId()`, `RandomCheckAccountNumber()`
- **Request builders** — `BuildCheckSaleRequest(Config)`, `BuildCheckDisburseRequest(Config)`, `BuildBankFundingSourceRequest(Config)`, `BuildActivityReportRequest(Config)`
- **Fraud response builders** — `BuildTmxResponse()`, `BuildMitekResponse()` for enrollment fraud scoring fields

4.4 Per-Test RestClient Pattern

Each test class creates and disposes its own `RestClient` — isolating connection state between classes:

```
[TestInitialize]
public void TestInit() { _client = CreateGatewayClient(); }

[TestCleanup]
public void TestCleanup() { _client.Dispose(); }
```

4.5 Response Code 400 = Success (Known Quirk)

Several Gateway endpoints return HTTP 200 with an internal `responseCode: 400` to indicate success. This is **not** an HTTP 400 Bad Request — it is a legacy convention from the underlying banking network protocol. Always assert both:

```
Assert.AreEqual(HttpStatusCode.OK, response.StatusCode);
Assert.AreEqual(400, result.ResponseCode, "responseCode 400 = Gateway success indicator");
```

Part 5: Test Coverage Summary

5.1 Tests by Domain

Domain	Test Class	Active Tests	Smoke	Ignored
Accounts	AccountInquiryTests	1	1	0
Accounts	AccountInquiryHistoryTests	3	1	0
Accounts	CombinedAccountInquiryTests	3	1	0
Accounts	EnrollNewCardholderTests	9	2	2
Accounts	PatronRetrievalTests	4	1	0
Accounts	UpdateAccountTests	4	0	2
Accounts	UpdateLimitTests	3	1	0
Accounts	VipAccountInquiryTests	6	1	0
Checks	CheckSaleTests	11	1	0
Checks	CheckDisbursementTests	4	1	0
Echo / Health	EchoTests + HealthTests	3	3	0
Finicity	FinicityTests	2	0	0
Funding	FundingBankTests	3	0	0
Funding	FundingCreditCardTests	3	0	1
Funding	FundingFinicityTests	3	0	1
Funding	FundingGenericTests	4	0	1
Funding	FundingPlaidTests	3	0	1
Merchants	MerchantTests	6	0	0
Merchants	AddVipSelectTransactionTests	4	0	0
Plaid	GetPlaidTokenTests	5	0	0
Reports	ActivityReportTests	11	1	0
Total	22 test classes	~104	~14	8

5.2 Ignored Tests and Reasons

Test Class	Count	Reason
EnrollNewCardholderTests	2	Finicity / Plaid sandbox requires separate setup
UpdateAccountTests	2	Pending stable test account

Test Class	Count	Reason
FundingCreditCardTests	1	Endpoint currently not used
FundingFinicityTests	1	Requires Finicity test environment setup
FundingGenericTests	1	Requires Finicity test environment setup
FundingPlaidTests	1	Requires Plaid sandbox token — coordinate with Dev

Part 6: Automation Gaps

The following Gateway API endpoints have wrapper methods in `GatewayApiFunctions` but are not called from any test:

Gap	Method	Endpoint	Priority
Bank Funding Add	AddFundingSourceBank	POST /api/funding/bank	High
Account Evaluation	EvaluateAccount	POST /api/evaluateaccount	Medium
CC Enrollment	EnrollNewCardholderCc	POST /api/accounts/newcardholder/cc	Medium
Devices — Create	CreateDevice	POST /api/devices	Low
Devices — Get	GetDevices	GET /api/devices	Low
Devices — Login	DeviceLogin	POST /api/devices/login	Low
Devices — Audit	DeviceAudit	POST /api/devices/audit	Low
Logging	CreateLog	POST /api/logging	Low

Highest priority gap: `AddFundingSourceBank` — the suite covers Remove and Update for bank funding sources but has no Add test.

Additional thin-coverage areas:

- **UpdateAccount** — 2 of 4 tests are ignored. Remaining 2 are positive-only; no invalid-field cases.

- **Merchant FeeTable / CalculateFees** — happy path only; no boundary or invalid-input coverage.
 - **Finicity ConnectUrl** — no Smoke tag and limited negative coverage.
 - **ActivityReport** — delivery method variants (email vs. download) not fully crossed with valid/invalid account combos.
-

Part 7: Key Concepts for New Engineers

What is the Services Gateway?

The Services Gateway is Pavilion's internal API layer between operator-facing systems (cage, kiosk, iGaming) and downstream banking networks. It handles:

- **New cardholder enrollment** — creates VIP card numbers and links funding sources
- **Account inquiry** — lookup patron by VIP card, SSN, patron ID, or history
- **Check sale and disbursement** — ACH check processing
- **Funding source management** — add, update, remove bank / credit card / Plaid / Finicity
- **Merchant profile and fee table** — operator-level configuration retrieval
- **VIP Select** — transaction recording for VIP program
- **Reports** — activity reporting with email or download delivery
- **Devices** — device management for kiosk and cage environments

responseCode 400 = Success

The most common source of confusion for new engineers. The Gateway's enrollment and check endpoints return HTTP 200 with `responseCode: 400` in the JSON body to indicate success. This is **not** an HTTP 400 Bad Request — it is a legacy convention from the underlying banking network protocol.

X-Activity-Id Header

Every request must include an `X-Activity-Id` header with a fresh GUID. This is used for distributed tracing and support ticket correlation. `GatewayApiFunctions` adds this automatically — do not add it manually in tests.

Part 8: Adding New Tests

Follow this pattern when writing a new test:

12. Create or identify the appropriate test class (matches the domain area)
13. Ensure the class inherits `BaseTest` with `[TestInitialize]` / `[TestCleanup]` for the `RestClient`
14. Use `TestDataGenerator` for random or reusable request values — do not hardcode PII
15. Call the relevant `GatewayApiFunctions.*` method — if the endpoint has no wrapper yet, add one following the 1:1 naming convention
16. Assert both HTTP status code and response body fields
17. Tag appropriately: `[TestCategory("SmokeTest")]` for the happy-path test of a new feature

Example skeleton:

```
[TestClass]
[TestCategory("API")]
[TestCategory("Regression")]
public class MyNewTests : BaseTest
{
    private RestClient _client = null!;

    [TestInitialize]
    public void TestInit() => _client = CreateGatewayClient();

    [TestCleanup]
    public void TestCleanup() => _client.Dispose();

    [TestMethod]
    [TestCategory("SmokeTest")]
    public void MyFeature_HappyPath_Returns200()
    {
        var body = new MyRequest { ... };
        var response = GatewayApiFunctions.MyEndpoint(_client, AccessToken, body);
        Assert.AreEqual(HttpStatusCode.OK, response.StatusCode, response.Content);
        var result = JsonConvert.DeserializeObject<MyResponse>(response.Content);
        Assert.IsNotNull(result.SomeField);
    }
}
```

Part 9: Resources & Next Steps

Resource	Location
Repo	dev.azure.com → PAV Products → Repos → services-gateway
Test project	/GlobalPayments.Gateway.Client.Tests.End2End
Config template	Configuration/test-keys.json
Local override	Configuration/test-keys.Development.json (obtain from team)
ADO Test Plans	dev.azure.com → PAV Products → Test Plans
Confluence QA Space	pavilionpayments.atlassian.net/wiki/spaces/QAQ

Recommended onboarding path:

18. Clone the repo and set up test-keys.Development.json
19. Build and run the SmokeTest suite — verify all 14 smoke tests pass
20. Read BaseTest.cs and GatewayApiFunctions.cs to understand the auth and API layer
21. Read one test class end-to-end (suggested: CheckSaleTests.cs) to see the full pattern
22. Pick one of the automation gaps from Part 6 and implement a test for it

Contact: brock.pivrotto@pavilionpay.com | QA Director: Joe Stelter